



Relatoría Técnica: Sincronización de Concurrencia en el Sistema de Gestión de Tráfico (SIGET)

Autor: Daniel Grisales

Programa: Tecnología en Desarrollo de Software, Institución Universitaria Pascual Bravo

Contexto y lógica de implementación

Para mitigar los riesgos de inconsistencia de datos en el Sistema de Gestión de Tráfico (SIGET), se implementó una simulación basada en el problema clásico del Productor-Consumidor. En este entorno virtualizado utilizando Python y la librería `threading`, los sensores de las intersecciones actúan como *productores* que inyectan lecturas de volumen vehicular en un búfer de memoria compartida. Simultáneamente, los módulos centrales de toma de decisiones actúan como *consumidores* que extraen estos datos para ajustar la semaforización en tiempo real.

La separación de responsabilidades garantiza que el hilo de captura de datos (Entrada/Salida) no se bloquee mientras otro hilo realiza el cálculo intensivo de análisis, mejorando drásticamente la responsividad del sistema.

Análisis técnico de mecanismos de sincronización

La gestión concurrente de este búfer compartido representa una sección crítica clásica. Para evitar condiciones de carrera (donde dos sensores intenten escribir en el mismo índice de memoria) y garantizar la fiabilidad del **SIGET**, se utilizaron dos mecanismos complementarios apoyados en el sistema operativo:

- **Mutex (Mutual Exclusion):** Se implementó un candado (`threading.Lock()`) que garantiza que solo un proceso (ya sea productor o consumidor) pueda alterar el estado de la lista compartida a la vez.
- **Semáforos Contadores:** Se utilizaron dos semáforos (`espacios_vacios` y `datos_disponibles`). Esto previene activamente dos errores catastróficos:
 - Desbordamiento: Si los sensores capturan datos más rápido de lo que el sistema los procesa, el semáforo de espacios detiene a los productores antes de sobrescribir datos vitales.
 - Inanición y Espera Activa: Si no hay tráfico, los consumidores entran en un estado de suspensión (bloqueo pasivo) gracias al semáforo de disponibilidad, evitando que la CPU se mantenga al 100% consultando un búfer vacío.

Durante la ejecución, se observó que la sincronización elimina por completo los interbloqueos (deadlocks), dado que el orden de adquisición de bloqueos (primero el semáforo, luego el mutex) es estricto y consistente, evitando la retención circular de recursos.

Reflexión y comprensión conceptual

La práctica desarrollada evidencia que, en el desarrollo de software moderno, la concurrencia es indispensable para aprovechar la arquitectura multinúcleo actual, pero introduce una enorme complejidad en la gestión del estado. El uso de hilos permite que

aplicaciones críticas como el SIGET mantengan alta disponibilidad frente a operaciones asíncronas de entrada/salida. Sin embargo, la desventaja de este enfoque recae en la dificultad de depuración; un error en la sincronización puede desencadenar fallos intermitentes no deterministas. Esta simulación conecta directamente con el objetivo central de los Sistemas Operativos: abstraer la complejidad del hardware para ofrecer primitivas robustas (como mutexes y semáforos) que nos permiten a los desarrolladores administrar el tiempo de CPU y la memoria de forma segura y altamente optimizada.